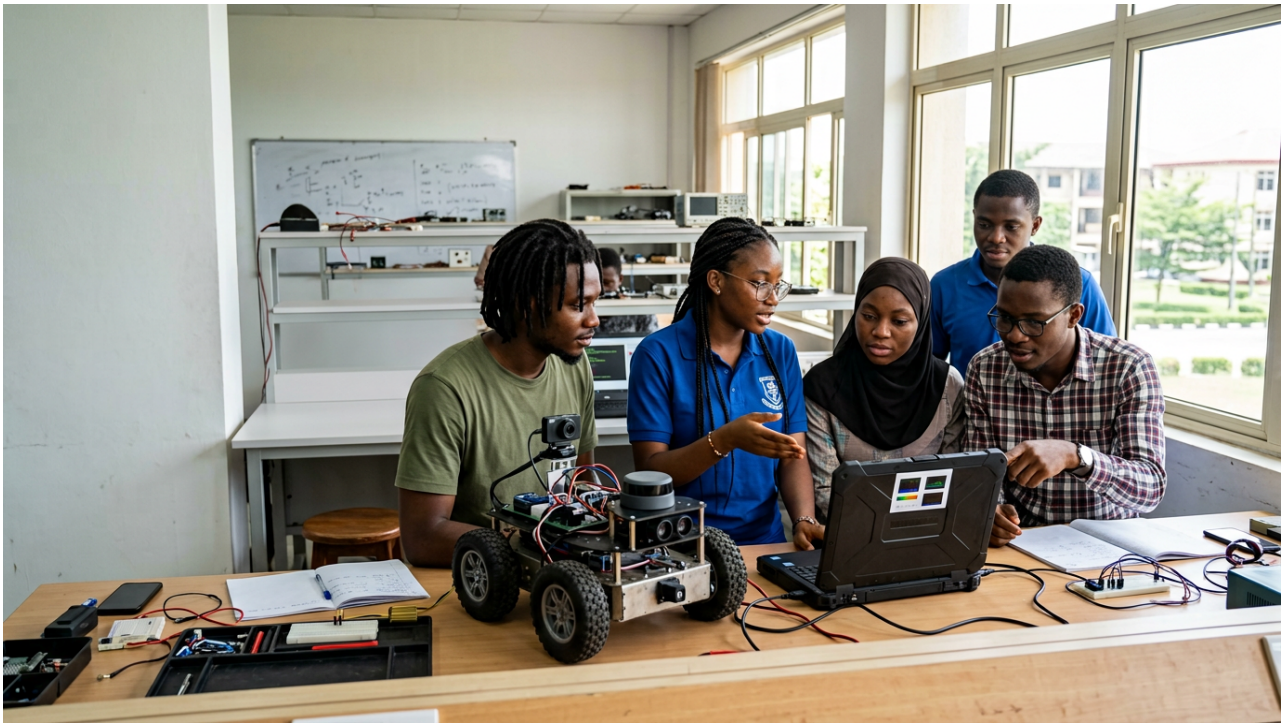
AI.101 — Introduction to Artificial Intelligence



Illustrative engineering-laboratory image; not documentary evidence of a specific university or event.

Narration manuscript · Module 9: Computer Vision and Machine Perception

Academic level: Bachelor of Artificial Intelligence and Robotics Engineering
Content policy: No faith-related content.

1. Seeing as a computational task

Computer vision studies computational methods for extracting useful information from images and video. It is tempting to describe vision as “teaching a computer to see,” but that phrase can conceal the actual tasks. A vision system may classify an image, locate objects, separate regions, estimate motion, identify a pose, or build a three-dimensional map. Each task has a different output and evaluation method.

A camera records light through an optical system and sensor. The resulting image is a grid of pixels. A color image commonly stores several channel values for each pixel, often red, green and blue, though other color spaces are useful. Pixel values are measurements affected by lighting, exposure, lens properties, sensor noise, motion and compression. The same object can look very different under different conditions.

A robot does not receive an abstract object label directly from the world. It receives sensor data, then software attempts to infer what may be present. The camera has a viewpoint and field of view. Objects can be occluded. Lighting can change. A model trained on one camera may not perform identically on another. These details are part of machine perception, not decorative setup.

2. From pixels to task outputs

Image classification assigns a label or scores to an entire image, such as “contains a person” or “contains a wrench.” It does not necessarily identify where the object is. Object detection predicts object categories and approximate locations, often represented by bounding boxes. Semantic segmentation assigns a class to each pixel, such as floor, wall or person. Instance segmentation separates individual objects of the same class. Tracking associates detections across frames to estimate how an object moves over time.

Pose estimation attempts to determine an object’s orientation or configuration. In robotics, a camera may estimate the position and orientation of a part so an arm can grasp it. A two-dimensional bounding box alone may not provide enough information for a safe grasp; depth, camera calibration, object geometry and uncertainty may be needed.

These tasks are related but not interchangeable. A classifier can correctly report that a person is in the image while failing to locate them. A detector can locate a box around an object without identifying its exact three-dimensional pose. The system designer must choose an output suited to the downstream decision.

3. Image preparation and classical methods

Before modern deep learning became widespread, many vision pipelines used hand-designed image-processing operations. A filter can smooth noise or emphasize edges. Thresholding can separate pixels based on intensity or color. Edge detectors identify rapid changes in image values. Feature descriptors summarize local patterns that may be matched across images.

These operations remain useful. A well-chosen threshold can be fast and interpretable for a controlled inspection task. A camera calibration procedure can correct geometric distortion. Classical methods can provide a simple baseline, help diagnose data, or operate within constrained hardware. The growth of deep learning has not made every previous method obsolete.

Preprocessing must be designed carefully. Resizing an image changes spatial detail. Normalizing pixel values affects numerical scale. Cropping can remove relevant context. Augmentation can help a model handle realistic variations, but unrealistic transformations may teach the wrong invariances. If the task depends on object orientation, rotating every image freely may destroy information that matters.

4. Training a vision model

A supervised vision model requires examples and labels. Labels may be image categories, bounding boxes, masks or keypoints. Creating them can be expensive and inconsistent. Human annotators may disagree on where an object boundary lies or whether a partially visible item should count as present. Annotation guidelines, review and quality checks are part of the data pipeline.

Split data by the intended generalization target. If images from the same video are randomly split across training and test, adjacent frames may be nearly identical, making test performance overly optimistic. If the model must work in a new building, test data should represent a building or camera condition not used during training. The evaluation should answer a realistic deployment question.

Metrics depend on the task. Classification may use precision, recall, confusion matrices and calibration. Detection may use overlap and precision-recall measures. Segmentation may compare predicted masks with reference masks. For a robot, a computer-vision metric should also be connected to task-level performance: did the robot identify the obstacle early enough to stop, and did it do so reliably under tested conditions?

5. Camera geometry and calibration

A camera maps a three-dimensional scene onto a two-dimensional image plane. This projection loses depth information unless additional cues, motion, stereo cameras or depth sensors are used. Camera calibration estimates parameters that describe the imaging geometry, including internal parameters such as focal length and lens distortion, and external parameters relating the camera to a robot or world coordinate frame.

Calibration matters because perception outputs must connect to action. If a robot arm uses a camera to estimate where an object lies, the coordinate transformation from camera frame to robot frame must be accurate enough for the task. A small calibration error can become a large grasp error at a distance. Calibration can also change if the camera is moved or its mounting shifts.

A monocular camera may infer depth from learned patterns or perspective cues, but these estimates depend on assumptions and training. A stereo pair estimates depth by comparing corresponding image points from two viewpoints. A depth camera measures range using its sensing technology, but may have its own range limits, reflective-surface failures and uncertainty. Sensor fusion can combine these sources, provided timing and coordinate frames are handled correctly.

6. Failure modes and domain shift

A vision model may fail on motion blur, glare, shadows, lens dirt, unusual object appearances, camera replacement, weather changes, occlusion or a background that differs from training data. This is an example of **domain shift**: the distribution of inputs encountered in use differs from the data used to develop the model.

A model may also rely on shortcuts. If every training image of a safety vest is photographed against a bright outdoor background, the network may use background cues rather than vest features. It can score highly on a test set that shares the same shortcut and then fail in a different environment. A useful evaluation deliberately varies nuisance conditions and examines failures rather than trusting a single aggregate score.

Confidence estimates can help prioritize review, but they are not automatically calibrated. A model may be very confident when wrong, especially on unfamiliar inputs. Systems can use out-of-distribution detection or uncertainty estimates, but these methods also have limitations. A robot should have a safe response when the vision system’s output is missing, low confidence or inconsistent with other sensors.

7. Worked example: detecting a corridor obstacle

Suppose a mobile robot must determine whether an obstacle occupies its near path. The camera supplies a frame. A detector identifies possible people, carts and furniture, along with locations and confidence values. The robot also uses lidar to estimate ranges. The perception module combines the image region with range measurements and transforms candidate positions into the robot coordinate frame.

The navigation system then reasons about clearance and predicted motion. A safety monitor checks whether the path remains within validated limits. If a person is detected near the path, the robot may reduce speed or stop. If the camera is obscured but lidar reports an object, the robot should still respond to the geometric evidence. If camera and lidar disagree, it may pause for another measurement rather than selecting the most convenient result.

Notice the chain: image measurement, object prediction, geometric estimate, navigation decision, control command and safety check. A failure can occur at every stage. The detector can miss an object; calibration can be wrong; the planner can use stale information; the motor controller can respond slowly. Evaluating the vision model alone cannot certify the complete robot.

8. Practical laboratory: image pipeline

Use Python and OpenCV to load a small image dataset. Inspect dimensions, color channels, sample examples and class counts. Implement a simple baseline, such as color thresholding or a feature-based classifier, where appropriate. Then train or evaluate a model for one clearly defined task. Keep test data separated by source sequence or environment when that better represents deployment.

Annotate sample outputs. For detection, draw boxes and confidence values. For segmentation, overlay a mask without hiding the underlying image. Evaluate false positives and false negatives, then inspect conditions associated with each. Test brightness changes, blur and partial occlusion. Do not present one successful screenshot as proof of robustness.

For a camera-based robotics exercise, estimate a known object’s image location and relate it to a simple tabletop coordinate frame. Record calibration assumptions and measurement error. Do not command a physical arm based solely on an unvalidated estimate; use simulation or a safe test setup with emergency stops and supervised operation.

9. Common mistakes

A common mistake is to treat image classifications as direct measurements of reality. They are model outputs based on data and assumptions.

Another is to use a benchmark split that leaks nearly identical frames into training and test. A third is to ignore camera calibration when converting pixels into physical coordinates. A fourth is to assume that a confidence score is a calibrated probability. A fifth is to select a metric that does not reflect the robot’s real task, such as celebrating high average accuracy while missing rare but dangerous obstacles.

10. Knowledge check and assignment

How does object detection differ from image classification? Why is camera calibration necessary for camera-guided manipulation? What is domain shift? Why can two sensors appear to agree while sharing a common failure? Which test conditions would you choose for a corridor obstacle detector?

For the assignment, build a small image-analysis pipeline and report the task definition, data source, annotation method, split strategy, baseline, metrics, failure analysis and limitations. Include at least one visual example of a correct prediction and one failure case. Propose a safe fallback for uncertain or unavailable vision outputs.

Module summary

Computer vision converts image measurements into task-specific estimates such as labels, object locations, masks, motion or pose. We reviewed image formation, classical preprocessing, learned models, camera calibration, evaluation and domain shift. We connected predictions to a robot’s navigation and safety architecture and emphasized that model outputs are not ground truth. In the next module, we will examine language systems and how to evaluate their outputs without confusing fluency with factual reliability.